MODULE 05 · DOMAIN 1 - DATA PREPARATION FOR ML AND AI

Feature engineering in practice

Encoding, scaling, imputation, binning, and the mistakes that quietly ruin models.

MODULE 05 of 43 DOMAIN 1 - Data Prep WEIGHT 28%

admin@sthithapragna.com

A better feature beats a better algorithm, **almost every time**.

The sequence, and why split comes first

01
Split
Train, validation, test. Before anything else touches the data.

02
Learn
Fit imputers, scalers and encoders on training data only.

03
Apply
Transform all three sets using what training taught you.

04
Persist
Save the fitted transformers. Inference needs exactly these.

ON THE EXAM

Splitting last instead of first is the most common serious mistake in this whole domain.

Data leakage is information in your features that will not exist at prediction time.

Four ways leakage gets in

PATTERN	WHAT IT LOOKS LIKE
Target leakage	A feature is a consequence of the outcome, not a cause of it
Train-test contamination	Scaler or imputer fitted on all data, so test statistics leak in
Temporal leakage	Rows from the future used to predict the past. Random split on time series
Duplicate leakage	The same record appears in train and test. Memorisation looks like skill

ON THE EXAM

Target leakage is the hardest to spot, because the offending feature usually looks perfectly reasonable.

How to find leakage before production does

- **Suspiciously high accuracy** is a symptom, not a success. Investigate it.
- Check **feature importance**. One feature dominating everything is a red flag.
- For each feature ask: **would I have this value at prediction time?** That question alone catches most of it.
- For time series, ask whether the value was **knowable at that timestamp**.
- Deduplicate before splitting, not after. Duplicates split across both sets defeat the split.

> ON THE EXAM
>
> One question, asked of every feature: will this value exist, with this meaning, at the moment I need a prediction?

Ask why it is missing before deciding what to do

- **Missing completely at random.** A sensor dropped a reading. Safe to impute.
- **Missing at random.** Missingness depends on another observed column.
- **Missing not at random.** The value is missing **because** of what it would have been.
- That third case is informative. Add a **was-missing indicator** column rather than hiding it.
- High earners declining to state income is the standard example. The blank means something.

> ON THE EXAM
>
> Imputing a value that is missing for a meaningful reason destroys the signal that the blank was carrying.

Strategies, and what each costs

STRATEGY	TRADE-OFF
Drop the rows	Simple. Loses data, and biases if missingness is not random
Drop the column	Right when most values are missing and the column is weak
Mean or median	Fast and common. Median resists outliers. Shrinks variance
Mode	For categoricals. Over-represents the most common class
Constant or 'Unknown'	Makes missingness explicit and visible to the model
Model-based, KNN or MICE	Most accurate. Slow, and can leak if fitted across the split

ON THE EXAM

Whatever you choose, fit it on training data only. A median computed over all data is contamination.

Finding them

- **IQR rule.** Below Q1 minus 1.5 times IQR, or above Q3 plus 1.5 times IQR.
- **Z-score.** More than about three standard deviations from the mean.
- Z-score assumes a roughly normal distribution. On skewed data it misleads.
- **Isolation Forest** or **Random Cut Forest** for multivariate outliers.
- A row can be unremarkable in every column and still be an outlier as a combination.

> ON THE EXAM
>
> Amazon SageMaker AI has Random Cut Forest as a built-in algorithm. It appears again in module 13.

Deciding what to do about them

- **Deleting is the reflex, and often wrong.** Ask whether it is an error or a real extreme.
- **Data error**, such as an age of 340: remove or correct it.
- **Genuine extreme**, such as a very large transaction: keep it. It may be the thing you predict.
- **Capping or winsorising** limits influence without discarding the row.
- **Log transform** compresses a long tail so extremes stop dominating.

> ON THE EXAM
>
> In fraud and anomaly detection the outliers are the positive class. Removing them removes the problem.

Normalisation or standardisation?

NORMALISATION, MIN-MAX

Squeeze into 0 to 1

- Bounded range, useful for neural networks.
- Preserves the shape of the distribution.
- Very sensitive to outliers.
- Use when bounds are known and stable.

STANDARDISATION, Z-SCORE

Mean 0, deviation 1

- Unbounded, so outliers do not compress everything.
- Assumes a roughly normal shape.
- The usual default for most algorithms.
- Use when outliers exist or bounds are unknown.

THE TIEBREAKER

Outliers present, or bounds unknown: standardise. Bounded input needed, as for many neural networks: normalise.

Which algorithms actually care about scale

- **Distance-based:** KNN, K-Means, SVM. Scale changes the answer completely.
- **Gradient-based:** neural networks, linear and logistic regression. Convergence suffers.
- **Regularised** models: penalties are applied unevenly across differently scaled features.
- **Tree-based models do not care.** Decision trees, random forests, XGBoost split on thresholds.
- Scaling before XGBoost is harmless and useless. It will not fix a poor model.

> ON THE EXAM
>
> 'We scaled the features and accuracy did not improve' on a tree model is expected behaviour, not a mystery.

Log and power transforms

- Income, prices and counts are usually **right-skewed**: a long tail of large values.
- A **log transform** compresses that tail and pulls the distribution towards normal.
- It also converts multiplicative relationships into additive ones, which linear models like.
- **Log of zero is undefined.** Use log of one plus x, or shift the data first.
- **Box-Cox** and **Yeo-Johnson** generalise this. Yeo-Johnson handles negatives.

> ON THE EXAM
>
> If you log-transform the target, remember to invert the transform on predictions before anyone reads them.

Encoding techniques, and when each fits

TECHNIQUE	USE IT WHEN
One-hot	No order, low cardinality. One binary column per category
Label or ordinal	A genuine order exists: small, medium, large
Binary	Moderate cardinality. Encodes n categories in log2(n) columns
Target or mean	High cardinality. Replaces category with its mean target. Leaks easily
Frequency or count	High cardinality. Replaces category with how often it appears
Hashing	Very high or unbounded cardinality. Fixed width, accepts collisions
Embeddings	Very high cardinality with meaningful similarity between categories

ON THE EXAM

Cardinality decides most of this. Count the distinct values before choosing anything.

Label encoding where there is no order

- Encoding red as 1, green as 2, blue as 3 tells the model that **green is between red and blue**.
- It also implies blue is three times red. Both statements are nonsense.
- Distance-based and linear models take that ordering literally, and it degrades them.
- **Tree models tolerate it better**, because they can split repeatedly on the same feature.
- Rule: order exists, label encode. No order, one-hot or something else.

> ON THE EXAM
>
> This is the most frequently examined encoding error, and it is stated almost word for word in questions.

The high-cardinality problem

- One-hot on 50,000 postcodes gives you **50,000 columns**, almost all zero.
- Memory explodes, training slows, and most models overfit the sparse columns.
- **Group the tail:** keep the top N categories and bucket the rest as 'Other'.
- **Target or frequency encoding** keeps it to one column.
- **Target encoding must be fitted on training folds only**, or it leaks the answer directly.

> ON THE EXAM
>
> A one-hot encoded column count in the tens of thousands is the signal. The fix is never a bigger instance.

Unseen categories at inference time

- A new postcode appears in production that was not in the training data. What happens?
- A naive one-hot encoder either errors, or silently produces an all-zero row.
- **Reserve an 'Other' or 'Unknown' category** during training, and map new values to it.
- Target encoders should fall back to the **global mean** for unseen values.
- Decide the behaviour deliberately. Left to chance, it fails on the day a new category arrives.

> ON THE EXAM
>
> This is the exact mismatch behind scenario 3 in module 04: two encoders disagreeing about a value neither had seen.

Binning, and why you would throw away detail

- **Binning** converts a continuous value into ranges: age becomes an age band.
- It captures **non-linear relationships** that a linear model could not otherwise express.
- It reduces the influence of outliers and of small measurement noise.
- **Equal-width** bins split the range. **Equal-frequency** bins hold equal counts.
- You are discarding information. Too few bins and real signal disappears.

> ON THE EXAM
>
> Binning helps linear models most, and helps tree models least, since trees find thresholds themselves.

Getting features out of a timestamp

- A raw timestamp is nearly useless. **Decompose it**: hour, day of week, month, is-weekend.
- **Elapsed time** features are often the strongest: days since signup, days since last order.
- Hour 23 and hour 0 are adjacent in reality and far apart as integers.
- **Cyclical encoding** fixes that: take the sine and cosine of the value.
- Holidays, paydays and business days often matter more than the raw date.

> ON THE EXAM
>
> Cyclical encoding is the detail most people miss. It applies to hours, days, months and compass bearings alike.

Turning words into numbers, the classical way

- **Tokenisation** splits text into units. Words, subwords or characters.
- **Bag of words** counts occurrences. Order is discarded entirely.
- **TF-IDF** weights terms down when they appear in many documents. Common words stop dominating.
- **N-grams** keep short phrases, so 'not good' survives as a unit.
- Stemming and lemmatisation collapse word forms. Stop-word removal drops filler.

> ON THE EXAM
>
> Embeddings replace all of this for modern AI work. Module 09 covers them. Both are examinable.

Advanced text preprocessing for AI workloads

- Classical NLP shrinks vocabulary. **Modern AI preprocessing preserves meaning instead.**
- Stemming and stop-word removal **hurt** embedding quality. They strip the context it needs.
- What matters now: cleaning boilerplate, normalising whitespace, fixing encoding, removing markup.
- **Subword tokenisation**, such as byte-pair encoding, handles unseen words without an unknown token.
- Token count drives cost directly on the FM path. Preprocessing is a cost lever, not just quality.

> ON THE EXAM
>
> The habits that helped bag-of-words actively harm embeddings. Unlearning that is part of moving to the FM path.

Sparse and dense representations

- One-hot and bag-of-words output is **mostly zeros**. Storing every zero wastes memory.
- **Sparse formats** store only non-zero positions and values. Often a hundredfold saving.
- RecordIO-protobuf supports sparse tensors, which is why it scales for wide feature matrices.
- Some algorithms need dense input. Converting sparse to dense is how memory errors happen.
- Embeddings are dense by design: few hundred dimensions, all meaningful.

> ON THE EXAM
>
> Sparse against dense is a memory decision, not a modelling one. It decides whether the job runs at all.

Derived and interaction features

- **Ratios** often beat their components: price per square metre, spend per visit.
- **Differences** capture change: this month against the rolling average.
- **Interactions** multiply features, letting linear models express 'A matters only when B'.
- **Aggregations** over a group are usually the strongest features in tabular ML.
- Aggregations computed over the whole dataset leak. Compute them within the training split.

> ON THE EXAM
>
> Domain knowledge lives here. A ratio a practitioner suggests will often outperform anything automated.

Correlated features and multicollinearity

- Two features carrying the same information add cost and no signal.
- In **linear models** it destabilises coefficients: they swing wildly and stop meaning anything.
- **Tree models are largely unbothered**, but importance gets split arbitrarily between the pair.
- Detect with a correlation matrix, or **variance inflation factor** for the formal version.
- Fix by dropping one, combining them into a ratio, or using PCA if interpretability is expendable.

> ON THE EXAM
>
> If a stakeholder asks why an obviously important feature scored low on importance, look for a correlated twin.

Feature selection and dimensionality reduction

- **Filter methods.** Rank by correlation or mutual information, then cut. Fast and model-agnostic.
- **Wrapper methods.** Recursive feature elimination. Accurate and expensive.
- **Embedded methods.** L1 regularisation drives weak coefficients to exactly zero.
- **PCA** creates new uncorrelated components. Powerful, and the components are not interpretable.
- PCA needs scaled input first, or the largest-magnitude feature dominates every component.

> ON THE EXAM
>
> If the scenario mentions explainability or regulation, PCA is usually wrong. You cannot explain a principal component to a regulator.

Deduplication and augmentation

- **Deduplicate first.** Exact duplicates inflate confidence and corrupt any split.
- Near-duplicates are harder and matter just as much. Fuzzy matching on keys helps.
- **Augmentation** creates plausible new examples: rotate, crop or flip images.
- For text: synonym swaps, back-translation. For audio: pitch, speed, background noise.
- Augment the **training set only**. Augmented rows in validation flatter the score.

> ON THE EXAM
>
> Augmentation is the honest way to grow a small dataset. It is also how you make a model robust to variation it will meet.

When the thing you predict needs work

- A right-skewed target, such as price, often trains better after a **log transform**.
- **Predictions then come back in log space.** Invert before anyone reads them.
- Metrics computed in log space are not comparable to metrics in the original units.
- For classification, the target needs encoding too, and the **positive class must be defined explicitly**.
- Regression targets can also be binned into classes when the business only needs bands.

> ON THE EXAM
>
> Forgetting to invert a target transform produces predictions that are quietly, confidently wrong by orders of magnitude.

Three ways to split, and when each is required

SPLIT	USE IT WHEN
Random	Rows are independent and classes are reasonably balanced
Stratified	Classes are imbalanced. Preserves class proportions in every split
Temporal	Anything with time. Train on the past, test on the future
Grouped	Multiple rows share an entity. Keep one customer entirely on one side

ON THE EXAM

Random splitting time series data is temporal leakage. It is the single most common split mistake on this exam.

Cross-validation, and the time series version

- **K-fold** splits into k parts, trains k times, and averages. Every row is tested exactly once.
- It gives a far more reliable estimate than one split, especially on small datasets.
- **Stratified k-fold** preserves class balance in each fold. The default for classification.
- **Standard k-fold is wrong for time series.** It trains on the future to predict the past.
- Use **rolling or expanding window** validation instead: always train before, test after.

> ON THE EXAM
>
> Cross-validation costs k times the compute. On a large dataset a single well-chosen split is often the sensible trade.

When to shuffle, and when not to

- Data often arrives sorted, by date or by class. Sorted order biases training badly.
- Training on all the negatives then all the positives makes gradient descent unstable.
- **Shuffle before splitting** when rows are independent.
- **Never shuffle time series before a temporal split.** You destroy the ordering you need.
- SageMaker AI can shuffle S3 keys across instances for distributed training.

> ON THE EXAM
>
> Shuffle and split are separate decisions. Shuffling first then splitting randomly is right for tabular data and wrong for time series.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Validation excellent, production poor	Leakage. Check features and how you split
Accuracy identical after scaling a tree model	Expected. Trees are scale-invariant
Training runs out of memory after encoding	One-hot on a high-cardinality column
Model does well on common cases, fails on rare	Class imbalance, or a non-stratified split
Distance model dominated by one column	That column is unscaled and larger in magnitude
Time series model good in test, bad live	Random split instead of temporal

ON THE EXAM

Six symptoms, six causes. Most feature engineering questions on this exam are one of these in a costume.

Documenting what you did, and why

- Record the **transformation**, its **parameters**, and the **data version** it was fitted on.
- A scaler is a fitted artefact, not code. Version it alongside the model, not beside the script.
- Feature definitions belong somewhere shared, so two teams cannot build the same feature twice.
- SageMaker Feature Store and model cards both hold parts of this. Modules 06 and 27.
- An undocumented feature is one nobody dares change, and one nobody can reproduce.

> ON THE EXAM
>
> Six months later the question is always the same: what exactly was done to this column, and can we do it again?

Fit on training. Apply everywhere. **Persist what you fitted**, because inference needs it too.

Where these transformations actually run

TECHNIQUE	WHERE YOU WOULD RUN IT
Scaling, encoding, imputation	Data Wrangler in Canvas, or scikit-learn in a Processing job
Joins, aggregations, deduplication	AWS Glue or EMR, at dataset scale
Profiling and outlier detection	DataBrew profile jobs, or Data Wrangler data insights
Bias metrics before training	SageMaker Clarify. See module 08
Serving the same features at inference	SageMaker Feature Store. See module 06
Anomaly and outlier scoring at scale	Random Cut Forest, a built-in algorithm

ON THE EXAM

The technique and the engine are separate choices. Module 04 chose the engine; this module chooses the technique.

A team builds a customer churn model. Validation accuracy is 97 percent, far above anything the business expected. In production the model performs barely better than guessing.

Reviewing the feature list, one column is 'account_closure_reason_code'. It is populated for churned customers and null for the rest. Feature importance shows it dominating every other feature.

What is the correct action?

A. Impute the nulls with a placeholder so the column is populated for all rows

B. Retrain with stronger regularisation so the model relies on it less

C. Remove the column, because its value does not exist at prediction time

D. Keep it, but reduce its weight using a feature importance penalty

◆ PAUSE AND DECIDE

What is the correct action?

A Impute the nulls with a placeholder so the column is populated for all rows

Makes the leak tidier, not smaller. The placeholder still marks 'has not churned', so the column still encodes the answer.

B Retrain with stronger regularisation so the model relies on it less

Regularisation reduces overfitting to noise. This feature is not noise, it is the target in disguise, so the model is right to trust it.

C Remove the column, because its value does not exist at prediction time

Textbook target leakage. A closure reason only exists after closure, so it can never be available when the prediction is needed.

D Keep it, but reduce its weight using a feature importance penalty

Negotiating with a feature that cannot exist at inference. At prediction time it will be null for everyone, and the model has learned to lean on it.

THE SIGNAL

Apply the one question: will this value exist, with this meaning, when I need a prediction? Here it cannot, by definition.

A logistics team is preparing features for a delivery-delay model. One column holds the destination postcode, with roughly 48,000 distinct values across the dataset.

After one-hot encoding, the training job fails with an out-of-memory error. The team asks whether they should move to a larger instance type.

What should you advise?

A Replace one-hot with target or frequency encoding, fitted on the training split only

B Move to a memory-optimised instance and keep the one-hot encoding

C Label encode the postcodes as integers from 1 to 48,000

D Drop the postcode column, since high-cardinality features are rarely useful

◆ PAUSE AND DECIDE

What should you advise?

A Replace one-hot with target or frequency encoding, fitted on the training split only

Collapses 48,000 sparse columns into one informative one, and the fitting caveat prevents the leak that target encoding invites.

B Move to a memory-optimised instance and keep the one-hot encoding

Buys hardware to hold 48,000 mostly-zero columns. Even if it fits, the model overfits sparse columns and training is far slower.

C Label encode the postcodes as integers from 1 to 48,000

Fits in memory and asserts postcode 48,000 is vastly greater than postcode 1. Postcodes have no ordering, so the numbers are fiction.

D Drop the postcode column, since high-cardinality features are rarely useful

Destination is likely one of the strongest predictors of delivery delay. The premise is wrong, not just the action.

THE SIGNAL

Note the phrase 'fitted on the training split only'. Target encoding uses the target, so fitting it across the split leaks the answer.

A team forecasts weekly energy demand from three years of hourly readings. They shuffled the rows, then split 80 percent for training and 20 percent for testing at random.

Test error is very low. Deployed against next week's data, error is roughly four times higher. No code changed between the two runs.

What is the underlying problem?

A The model is underfitting and needs more capacity

B The features were not scaled before training

C The test set was too small to estimate error reliably

D Shuffling and random splitting leaked future readings into training

◆ PAUSE AND DECIDE

What is the underlying problem?

A The model is underfitting and needs more capacity

Underfitting shows as poor performance on training and test alike. Test error here was very low, so the model learned the training distribution well.

B The features were not scaled before training

Scaling affects convergence and distance calculations. It would not produce a low test error followed by a fourfold live error.

C The test set was too small to estimate error reliably

Twenty percent of three years of hourly readings is over five thousand rows. Sample size is not the issue.

D Shuffling and random splitting leaked future readings into training

A random split puts Tuesday in training and Wednesday in test. The model interpolates between known neighbours instead of forecasting forward.

THE SIGNAL

For anything time-ordered, split chronologically. The test set must sit entirely after the training set, exactly as production does.

> A team trains an XGBoost model to predict equipment failure. Accuracy has plateaued below the target. A colleague proposes standardising all numeric features, arguing that unscaled features are holding the model back.
>
> The features include operating hours ranging from 0 to 90,000 and temperature ranging from 15 to 95.

How should you respond?

A Agree. The 90,000 to 95 range difference will dominate the model's splits

B Explain that XGBoost is scale-invariant, and look for the real cause elsewhere

C Agree, but use min-max normalisation instead, as it preserves distribution shape

D Agree, and additionally apply PCA to remove correlation between the features

◆ PAUSE AND DECIDE

How should you respond?

A Agree. The 90,000 to 95 range difference will dominate the model's splits

True for distance-based models, false for trees. A tree evaluates thresholds within one feature at a time, so cross-feature magnitude is irrelevant.

B Explain that XGBoost is scale-invariant, and look for the real cause elsewhere

Trees split on thresholds inside a single feature. Rescaling moves the threshold and leaves every split decision identical.

C Agree, but use min-max normalisation instead, as it preserves distribution shape

Debating which ineffective transform to apply. Neither changes a tree's behaviour, and min-max is the more outlier-sensitive of the two.

D Agree, and additionally apply PCA to remove correlation between the features

Adds a transform that destroys interpretability on a maintenance model, to fix a problem that does not exist.

THE SIGNAL

Scaling before a tree model is harmless and useless. Recognising work that will not help is as valuable as knowing what will.

What to take from this module

- **Split first, fit on training only, persist what you fitted.** Most errors here violate one of these.
- **Leakage test:** will this value exist, with this meaning, at prediction time?
- **Trees ignore scale.** Distance and gradient models do not.
- **Cardinality decides encoding.** One-hot is for low cardinality only.
- **Time series means a temporal split.** Random splitting time data leaks the future.

> ON THE EXAM
>
> Module 06 covers the Feature Store, which exists to solve the persistence and skew problems named here.